Project code: chat-validation

The code for this example project is in the ch09/chat-validation folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch09/chat-validation
```

You'll need either the OpenAI API or the Google API keys to access their REST API. You'll also need Upstash Redis REST URL and the Upstash Redis REST token as well. The appendix offers instructions on how to set up and create a secret key for OpenAI and other providers:

```
// .env
GEMINI_API_KEY=<GEMINI_AI_SECRET_KEY>
UPSTASH_REDIS_REST_URL=<UPSTASH_REDIS_REST_URL>
UPSTASH_REDIS_REST_TOKEN=<UPSTASH_REDIS_REST_TOKEN>
```

Instructions on how to set up and create secret keys for Google and Upstash Redis are also provided in the appendix.

The following listing shows how to validate incoming requests using Zod.

Listing 9.1 Validating incoming requests using Zod

```
import { z } from 'zod';

const promptSchema = z.object({
  prompt: z.string().min(1).max(1000),
});

export default async function POST(req, res) {
  try {
    const validatedData =
promptSchema.parse(req.body);

    const response =
await process(validatedData.prompt);
    res.status(200).json({ response });
  } catch (error) {
    return res.status(400).json({ error: error.errors });
  }
}
```

Imports the Zod schema validation library

Defines a Zod schema for validating the structure of incoming data

Attempts to validate the incoming request body against the defined Zod schema

After successful validation, this line processes the validated prompt.

With this check, if a user submits a POST request with more than 1,000 characters, then the server would reject the request altogether. Additional surface-level checks can also be implemented at this point to ensure some basic sanity tests for your inputs.